四边形不等式优化

1. 背景

- 四边形不等式(quadrangle inequality)应用于DP优化，起源于1971年Knuth(高纳德)的一篇论文，用来解决最优二叉搜索树问题。
- 1980年，储枫(F.Frances Yao，姚期智的夫人)做了深入研究，扩展为一般性的DP优化方法，把一些DP问题从$O(n^3)$优化到$O(n^2)$。

2. 应用场合

有一些常见的DP问题，通常是区间DP问题，它的状态转移方程是

$$dp[i][j] = min\{dp[i][k] + dp[k+1][j] + w[i][j]\}$$

其中，$i \le k < j$，初始值$dp[i][i]$已知，min()也可以是max()

- (1) $dp[i][j]$表示从i状态到j状态的最小花费。题目一般是求$dp[1][n]$，即从起点到终点n的最小花费。
- (2) $dp[i][k] + dp[k+1][j]$ 体现了递推关系。k在i和j之间滑动，k有一个最优值，使得$dp[i][j]$最小。
- (3) $w[i][j]$是和题目有关的费用，如果它满足四边形不等式和单调性,那么计算时就能进行四边形不等式优化。

这类问题的经典的例子就是“石子合并”。它的状态定义就是上面的$dp[i][j]$，表示区间$[i,j]$上的最优值，$w[i][j]$是从第i堆石子合并到第j堆石子的总数量。

2.1 石子合并

问题描述

有n堆石子排成一排，每堆石子有一定的数量。将n堆石子合并成一堆，每次只能合并相邻的两堆石子，合并的花费为这两堆石子的总数。经过$n-1$次合并后成为一堆，求最小总花费。

输入

第1行输入整数n，表示有n堆石子。第2行输入n个数，分别表示这n堆石子的书目。

输出

最小总花费。

输入样例

```
3
2 4 5
```

输出样例

```
17
```

思路

下面用DP求解。

定义$dp[i][j]$为合并第i堆到第j堆的最小花费。

状态转移方程为

$dp[i][j] = min\{dp[i][k] + dp[k+1][j] + w[i][j]\}$，$i \le k < j$

$dp[1][n]$就是答案，方程中的$w[i][j]$是从第i堆石子合并到第j堆石子的总数量。

按自顶向下的思路分析状态转移方程，很容易理解。计算大区间$[i, j]$的最优值时，合并它的两个子区间$[i, k]$和$[k+1, j]$，对所有可能的合并（$i \le k < j$，即k在i和j之间滑动）采用最优的合并。子区间再分解为更小的区间，最小的区间$[i, i+1]$只包含两堆石子。

编程采用自底向上递推的方法，先在小区间进行DP得到最优解，然后再逐步合并小区间为大区间。

```
for(int i=1;i<=n;i++) dp[i][i]=0; //初始值
for(int len=2;len<=n;len++) //len为区间[i,j]的长度，从小区间扩展到大区间
    for(int i=1;i<=n-len+1;i++){ //区间起点i
        int j=i+len-1;//区间终点j，i<=j<=n
        for(int k=i;k<j;k++) //大区间[i,j]从小区间[i,k]和[k+1,j]转移而来
            dp[i][j]=min{dp[i][j],dp[i][k]+dp[k+1][j]+w[i][j]};
    }
```

代码中有三重循环，复杂度为$O(n^3)$。

3. 四边形不等式优化操作

只需一个简单的优化操作，就能把上面代码的时间复杂度降为$O(n^2)$。这个操作就是把循环 $i \le k < j$ 改为 $s[i][j-1] \le k \le s[i+1][j]$。 $s[i][j]$ 记录$i \sim j$的最优分割点，在计算$dp[i][j]$的最小值时得到区间$[i,j]$的分割点k，记录在 $s[i][j]$ 中，用于下一次循环。

这个优化称为四边形不等式优化。下面给出优化后的代码。

```
for(int i=1;i<=n;i++) {
    dp[i][i]=0;
    s[i][i]=i;      //初始值
}
for(int len=2;len<=n;len++)
    for(int i=1;i<=n-len+1;i++){
        int j=i+len-1;
        for(int k=s[i][j-1];k<=s[i+1][j];k++)   //缩小循环范围
            if(dp[i][j]>dp[i][k]+dp[k+1][j]+w[i][j]){ //是否更优
                dp[i][j]=dp[i][k]+dp[k+1][j]+w[i][j];
                s[i][j]=k;  //更新最佳分割点
            }
    }
```

代码的**复杂度**如何?

代码中i和k这两个循环，优化前复杂度为$O(n^2)$。

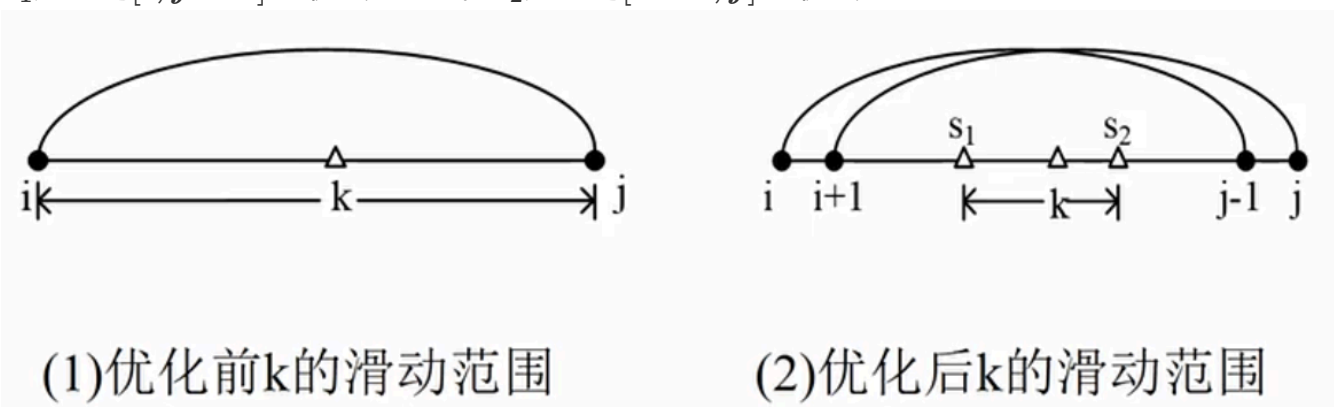
优化后，每个i内部的k的循环次数为$s[i+1][j]-s[i][j-1]$，其中$j=i+len-1$。那么:

$i=1$时，k循环$s[2][len]-s[1][len-1]$次;
$i=2$时，k循环$s[3][len+1]-s[2][len]$次;
...
$i=n-len+1$时，k循环$s[n-len+2][n]-s[n-len+1][n+1]$次;

将上述次数相加，总次数为
$s[2][len]-s[1][len-1]+s[3][len+1]-s[2][len]+...+s[n-len+2][n]-s[n-len+1][n+1]=s[n-len+2][n]-s[1][len-1]<n$

i和k循环的时间复杂度优化到了$O(n)$，总复杂度从$O(n^3)$优化到了$O(n^2)$。

s_1是区间$[i,j-1]$的最优分割点，s_2是区间$[i+1,j]$的最优分割点。



(1)优化前k的滑动范围　　(2)优化后k的滑动范围

看到这里我们可能会产生两个疑问：

（1）为什么能够把 $i\le k<j$ 缩小到$s[i][j-1]\le k\le s[i+1][j]$?
（2）$s[i][j-1]\le s[i+1][j]$成立吗?

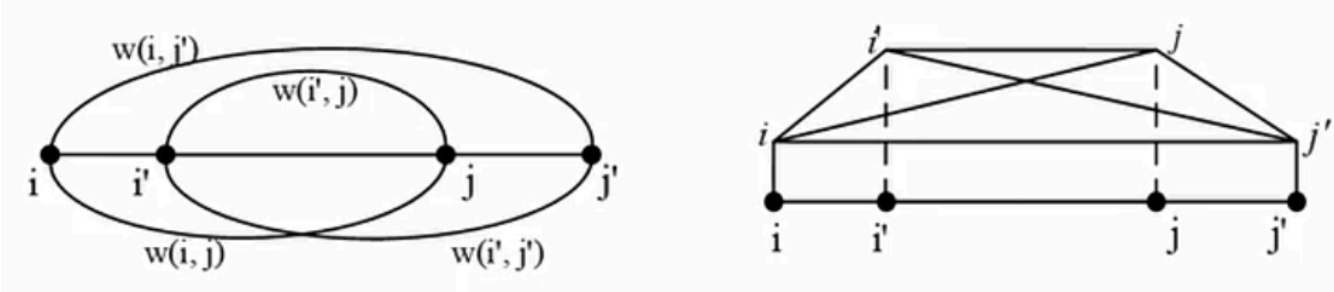
4. 四边形不等式定义和单调性定义

在四边形不等式DP优化中，对于费用w，有两个关键内容：四边形不等式定义和单调性。

4.1 四边形不等式定义 1

设w是定义在整数集合上的二元函数，对于任意整数$i\le i'\le j\le j'$，如果有$w(i,j)+w(i',j')\le w(i,j')+w(i',j)$，称$w$满足四边形不等式。

四边形不等式可以概括为：两个交错区间的w的和小于或等于小区间与大区间的w的和。



为什么称为“四边形”，参照右图把它画成平行四边形$i'ij'j$。图中对角线长度和$ij+i'j'$大于平行线长度和$ij'+i'j$，这与四边形不等式的几何性质是相反的，所以可以理解成“反四边形不等式”。这个“四边形”只是一个帮助理解的示意图，并没有严谨的意义。当中间两点$i'=j$时（两点重合），四边形变成了三角形。

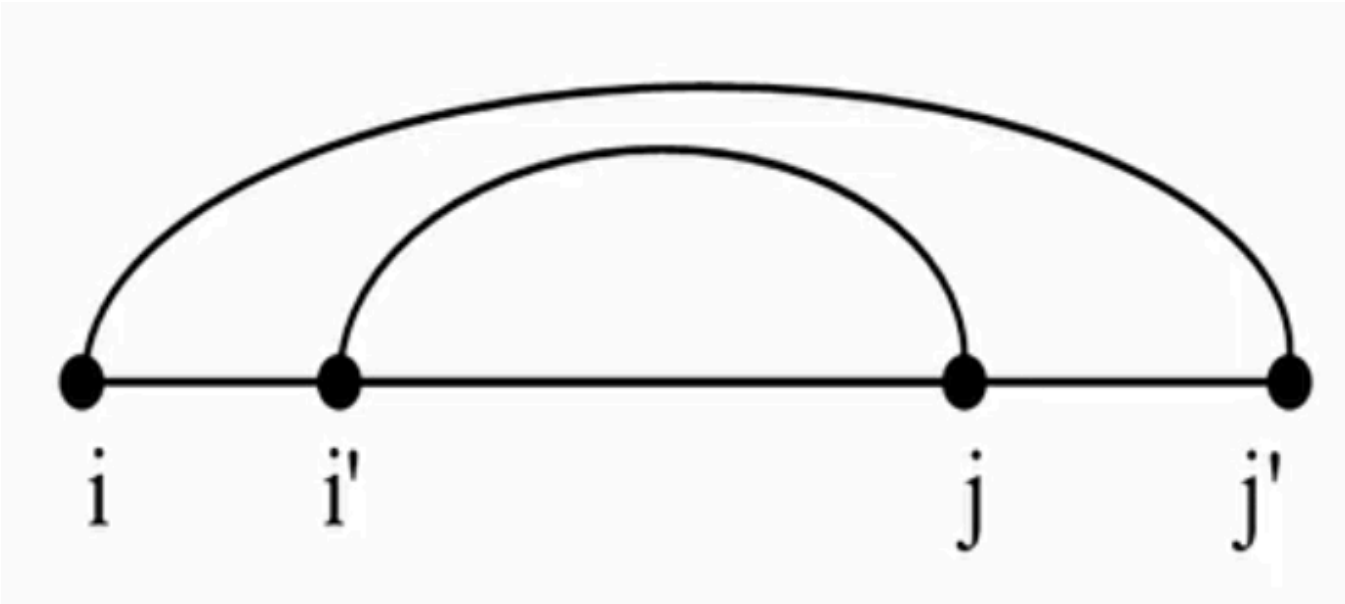
4.2 四边形不等式定义2

对于整数$i<i+1\le j\le j+1$，如果有$w(i,j)+w(i+1,j+1)\le w(i,j+1)+w(i+1,j)$，则称$w$满足四边形不等式。

4.3 单调性

设w是定义在整数集合上的二元函数，如果对任意整数$i\le i'\le j\le j'$，有$w(i,j')\ge w(i',j)$，称w具有单调性。

单调性可以形象地理解:如果大区间包含小区间，那么大区间的w值大于等于小区间的w值。



在石子合并问题中，令$w[i][j]$等于从第i堆石子合并到第j堆石子的石子总数，它满足四边形不

等式的定义、单调性。

- $w[i][j]+w[i'][j']=w[i][j']+w[i'][j]$,满足四边形不等式定义。
- $w[i][j'] \geq w[i'][j]$，满足单调性。

利用w的四边形不等式、单调性的性质，可以推导出四边形不等式定理，用于DP优化。

5. 四边形不等式定理

四边形不等式定理 :如果$w(i，j)$满足四边形不等式和单调性，则用DP计算$dp[][]$的时间复杂度是$O(n^2)$的。

- **引理1**:状态转移方程$dp[i][j]=min(dp[i][k]+dp[k+1][j]+w[i][j])$，如果$w[i][j]$满足四边形不等式和单调性，$dp[i][j]$也满足四边形不等式。
- **引理2**:记$s[i][j]=k$是$dp[i][j]$取得最优值时的k，如果$dp[i][j]$满足四边形不等式,有$s[i][j-1] \leq s[i][j] \leq s[i+1][i]$,即$s[i][j-1] \leq k \leq s[i+1][j]$。

引理2直接用于DP优化，复杂度为$O(n^2)$。

详细证明可参见《算法竞赛》5.10.4 四边形不等式定理 P380